Design, Development and Testing of BlockXAI-SecOps: A Secure SDLC Architecture Integrating Blockchain and Explainable AI

***Abstract*—In the current world of growing security threats in sectors like finance and healthcare, secure software systems are of utmost importance because the current Software Development Life Cycle (SDLC) lacks traceability and transparency and has risks of being tampered with. In addition, another problem current SDLCs have is that despite their modernization by using AI, there is little to no understanding on how they make decisions and form conclusions using AI thus creating a black box-like effect where both the client and the developer are not able to explain the SDLC's choices. In this paper we look into a new SDLC that uses Blockchain and Explainable AI(XAI) to address these problems, where blockchain ensures transparency and immutability of designs, while XAI ensures that both client and developer have a clear understanding of the decisions made by AI.**

***Index Terms*—Blockchain, Explainable AI, Secure SDLC, Hyperledger Fabric, Requirement Analysis, Quality Gates**

I. Introduction

Software is not just a tool anymore it is an integral part of everything we do, and is impossible to do any task in any other field like finance, healthcare, education, infrastructure etc without using software. Due to the importance of software in all aspects of our life, it is present all around us and the bigger the reach the bigger the risk it poses when it fails. In recent years we have seen and learnt the vulnerabilities of software due to various supply chain failures where hackers plant malware into the software which then gets updated to everywhere in the world using that particular software. This could occur from anywhere like a single library with hidden vulnerabilities or a security tool making calls nobody can fully explain. Traditional software Development Life Cycle (SDLC) models like waterfall, agile or even DevSecOps were never designed to handle security breaches causing incomplete audit trails and unexplainable warnings generated by AI that nobody can understand. These models don't guarantee transparency and leave organizations struggling with accountability when trust matters the most.

To address these problems, we propose BlockXAI-SecOps, a new way of running the software development life cycle that combines blockchain with explainable AI (XAI). Blockchain gives us an unchangeable or immutable, tamper proof log of every requirement, design, code, commit and test. It also provides multi signature approval in the form of smart contracts and programmable quality gates to ensure that no step has been bypassed. Meanwhile XAI solves the "black box" problem of using modern AI tools which do not explain why or how they came to a particular solution or decision for a problem. Instead of dropping unexplained alerts, our system explains its reasoning in a simple human readable language thus helping developers and clients understand why decisions were made. Together blockchain and XAI make development not just more secure but also more transparent and understandable for both developers and stakeholders who may not have a technical understanding of the problem.

In this paper we will be focusing on the requirement analysis phase because mistakes and misunderstanding of the requirements for any project can have a cascading effect on the entire project resulting in increased costs to correct these mistakes at a later stage. With BlockXAI-SecOps, every requirement is written onto the blockchain and locked so it cannot be tampered with during the development process. At the same time XAI tools analyse these requirements for conflicts or ambiguities and explains them in clear terms so there is no misunderstanding between developers and clients or stakeholders. By integrating immutability and interpretability this approach gives confidence and trust for the usage of these software systems in any sector like finance, healthcare or education.

The rest of this paper is organized as follows: Section II provides a review of work done in related problem domains. Section III outlines the overall methodology , followed by the Experimental work done in accordance with the proposed system in section IV. The results of this experimental work along with its detailed analysis can be found in section V, which if followed by section VI which has the final concluding remarks along with the future scope of work.

II. Literature Review

Recent work in software engineering has increasingly explored applying blockchain and explainable AI (XAI) to improve trust, traceability, and automation in the Software Development Life Cycle (SDLC). However, most studies treat blockchain and explainable AI separately. Only a few provide an integrated view that enforces both immutability and interpretability throughout the SDLC. Below we synthesize the literature across three themes: blockchain for SDLC, XAI for software engineering, and early efforts that combine the two.

A. Blockchain for SDLC and DevSecOps

Blockchain has been investigated primarily as an infrastructure for tamper-evident logging and artifact traceability.

TABLE I
REPRESENTATIVE WORKS ON BLOCKCHAIN AND XAI FOR SDLC (SELECTED)

Author(s), Year	Focus	Method	Key contribution	Limitations
Marjanović et al. (1), 2023	Blockchain	Permissioned ledger for compliance tracking	Immutable audit trail for security requirements	No AI/XAI integration; manual steps remain
Akbar et al. (2), 2022	DevOps security	Blockchain in CI/CD pipeline	Tamper-evident build/deploy logs	Scalability and latency concerns
Al-Breiki et al. (3), 2020	Oracles/trust	Comparative study on oracles	Taxonomy and trust issues for off-chain data	Conceptual; limited implementations
Song et al. (4), 2019	Patch management	Hyperledger Fabric prototype	Tamper-proof patch delivery	Focused on maintenance phase
Elapolu et al. (5), 2024	Requirements traceability	On-chain requirement records	Immutable requirement history	Lacks automated conflict detection
Arora et al. (6), 2025	XAI in SDLC	Survey/taxonomy	Phase-wise XAI applicability mapping	Conceptual; limited empirical studies
Tritscher & Kern (7), 2023	XAI for anomalies	Post-hoc feature relevance methods	Evaluation of explainability techniques	Not tailored for SDLC artifacts
Gecer & Bener (8), 2025	Defect prediction XAI	Explainable ML models	Interpretable defect prediction	Dataset dependence; limited generalization
Basheer et al. (9; 10), 2025	Vulnerability detection XAI	LLM + hybrid explainers	LLM-driven vulnerability detection with explanations	Validation limited to small corpora
Himdi (11), 2024	Blockchain + AI	Architectural framework (smart cities)	Integration blueprint for AI + ledger	Domain specific; not SDLC focused
Xu et al. (12), 2024	Trustworthy AI	Survey of lifecycle approaches	Discussion of anchoring AI artifacts on chain	High level; lacks implementation details
Kuznetsov (13), 2024	Integration patterns	Practical integration guidance	Guidance for embedding AI and blockchain	Implementation and evaluation limited
Qureshi (14), 2024	DevOps + DLT	Exploratory study	Roadmap for integrating DLT into DevOps	Preliminary; needs prototypes
Jovanovic (15), 2024	Federated/XAI	Blockchain anchoring for FL/XAI	Robustness for federated explainable models	Early stage research
Meinen (16), 2024	Component lifecycle	Blockchain for component provenance	Tracking components across supply chains	Focus limited to supply chain provenance
Proposed work	**Secure SDLC**	**Blockchain + XAI + smart gates**	**Unified, anchored explanations + enforcement across SDLC**	**Performance and operational tradeoffs**

Marjanović et al. (1) demonstrated how permissioned ledgers can securely track compliance requirements, giving immutable audit trails that are useful for regulatory verification. Subsequent works extended this idea to DevOps pipelines: Akbar et al. (2) and Saleh (17) propose blockchain-backed CI/CD logging to harden build and deployment records, while Song et al. (4) used Hyperledger Fabric for secure patch distribution. Research on blockchain oracles and trustworthiness by Al-Breiki et al. (3) highlights the importance of reliable off-chain data when integrating external evidence with blockchains. Several surveys and system papers argue that blockchain increases accountability in software processes but also identify practical limitations such as storage costs, latency and scalability for large projects (13; 18; 19).

B. Explainable AI in Software Engineering

Parallel to blockchain work, XAI research focuses on making automated analyses — defect prediction, anomaly detection, and vulnerability identification — interpretable to developers and stakeholders. Tritscher and Kern (7) evaluated post-hoc feature relevance techniques for anomaly detection, and Arora et al. (6) provided a taxonomy of XAI approaches applicable across SDLC phases. Gecer and Bener (8) and Basheer et al. (9; 10) examined explainable models for defect and vulnerability detection, showing that explanations improve developer trust and aid debugging. However, XAI studies frequently emphasize model interpretability but rarely address the integrity of the explanations, which is a gap that weakens auditing when decisions are verified later on in the lifecycle.

C. Initial Blockchain + XAI Convergence

A smaller set of works consider blockchain and AI together. Himdi (11) and Xu et al. (12) have dicussed about architectures that combine immutable ledgers with AI analytics for domains such as smart cities, while Jovanovic (15) explores blockchain integration with XAI settings. Kuznetsov (13) and Qureshi (14) provide practical discussions on integrating distributed ledgers into development toolchains. These studies show promise: blockchains can anchor AI outputs (hashing models, explanations or audit artifacts) to provide tamper evidence. Still, most prior proposals are conceptual or domain-specific, lacking an end-to-end SDLC design that (a) embeds XAI at decision points, (b) anchors XAI outputs on-chain, and (c) enforces automated quality gates via smart contracts.

D. Synthesis and Observed Gaps

From the surveyed literature we observe three recurring strengths and three key gaps:

Strengths: (1) blockchain solutions reliably provide immutable provenance for artifacts and operations (1; 4), (2) XAI techniques materially improve developer understanding and trust for automated analysis (6; 7; 8), and (3) combining ledger anchoring with AI outputs is technically feasible and conceptually attractive (11; 12).

Gaps: (1) *Lack of integration*: few works present a coherent, phase-spanning SDLC that jointly uses blockchain and XAI for requirements, design, implementation, and testing. (2) *Explanation provenance*: XAI research rarely secures the integrity of explanations (i.e., ensuring explanations themselves are auditable and untampered). (3) *Operational concerns*: real deployments must resolve storage, scalability, and latency trade-offs when anchoring large amounts of XAI and development metadata on chain (3; 13; 19).

III. Methodology

The BlockXAI-SecOps framework integrates explainable artificial intelligence (XAI) with blockchain technology throughout the SDLC to enhance security, transparency, and auditability. The methodology comprises six phases, each incorporating automation, human oversight, and immutable record-keeping.

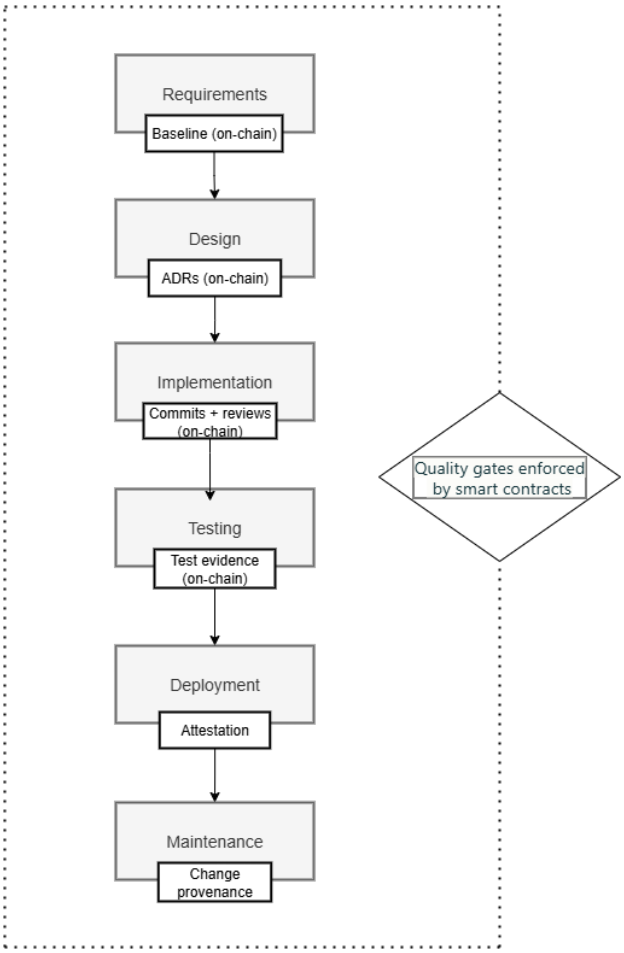

Fig. 1. Phases of the SDLC in BlockXAI-SecOps

A. Phase 1: Secure Requirements Analysis

Teams set out to collect all essential expectations for the system—what it should do (functional requirements) and how well it should perform (non-functional requirements) . To keep things consistent from the beginning, every team member logs into a secure portal and submits requirements, which are immediately cleaned up and standardized by language processing tools, so nothing gets lost in translation. Instead of handling this all by hand, modern systems use BERT, a powerful language model, to transform plain English requirements into meaningful data points. This helps detect conflicts in ideas—like when one requirement contradicts another.

B. Phase 2: XAI-Enhanced Design

In this, the security architects focus on risks by applying a STRIDE strategy, a well known framework for threat evaluation like spoofing, information leaks, data tampering, DOS, or unauthorized privilege. Everything is documented: access information, information flow, technology stacks used, and defenses that have been put.

XAI threat analysis module is responsible for analyzing designs against known threats by examining encryption and authentication mechanism, logging capability, and input validation. Computation of risk scores, and explainability analysis identifies factors that contribute towards risk. Designers receive clear explanations highlighting security gaps with actionable recommendations.

C. Phase 3: Blockchain-Secured Implementation

When developers commit code, Git commit hashes are submitted to blockchain for tracking. Also, automated tools detect any fresh changes to the code, while ML-XAI Models scan code features like confusing variable names, risky complexity, or misuse of encryption and input validation. If a serious issue shows up, the system automatically pauses and requires security officer's approval and justification. Smart contracts enforce these policies ensuring no security issues.

D. Phase 4: XAI-Driven Testing

Systematic test case are generated using threat models from the design phase. Security engineers develop tests for each identified threat. Pentesting discovers vulnerabilities with root cause analysis that also gives recommended fixes to those. Test artifacts, coverage metrics, and explanations are stored on the Test Ledger, recording test details, expected and actual behavior, pass or fail status, and explanations.

E. Phase 5: Secure Deployment

Deployment environments are prepared where configuration files are hashed and compared against the baselines. Any deviation from it trigger the system requiring explicit approval. Smart contracts enforce policies required and verify compliance of these as well.

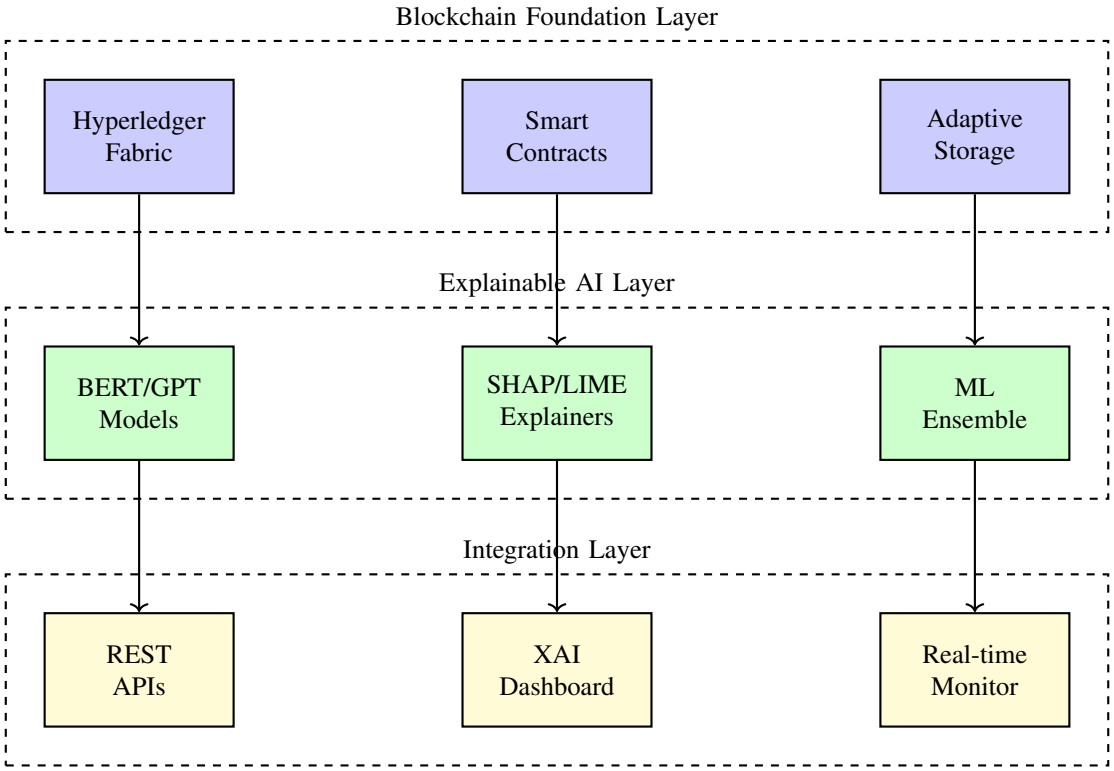

Fig. 2. BlockXAI-SecOps Three-Layer System Architecture

F. Phase 6: Continuous Maintenance

All systems in the production phase generate logs on HTTP response times, error rates, resource utilisation, database performance and authentication patterns. Many anomaly detection models are trained on historical behaviour and can identify deviations from the baseline. When these anomalies are detected, the XAI analysis can explain the causes ranked by probability allowing teams to differentiate between actual attacks, credential exposure and legitimate user behaviour changes. Recommended actions are then given based on the analysis to decide the best response.

IV. Experimental Work

To validate the feasibility and effectiveness of BlockXAI-SecOps, we implemented a modular prototype integrating blockchain infrastructure, semantic requirement analysis, machine learning risk models, and explainability mechanisms.

A. Environment Setup

1) Blockchain Setup: We used Hyperledger Fabric 2.5 with three organizations: *Development*, *Audit*, and *Client* and each had one peer node. Chaincode was written in Go to record requirement hashes, explanation references, and multi-signature approvals. Large artifacts were stored off-chain, anchored using SHA-256 hashes to ensure immutability.

2) NLP for Requirement Semantics: The NLP layer leveraged the ‘bert-base-uncased‘ model to encode requirements into 768-dimensional embeddings. Cosine similarity was computed to detect near-duplicates or contradictory requirements. Ambiguity detection relied on heuristics (e.g., modal verbs, vague quantifiers) and a BERT-based binary classifier trained on annotated requirement samples.

3) Risk Scoring Models: An ensemble of RandomForest, Gradient Boosting, and XGBoost classifiers was trained on feature sets including historical bug records, commit metadata, requirement complexity, and change frequency. A weighted majority voting mechanism produced final predictions.

4) Explainability Integration: We incorporated SHAP for computing global feature importances and LIME for generating interpretable local explanations for individual predictions. Each explanation was formatted in JSON, cryptographically hashed, and stored immutably on the blockchain for auditability.

B. System Context and Stakeholders

- **School Principal:** Policy definition—specifying access rights for students, parents, teachers.
- **IT Security Officer:** Compliance—ensuring GDPR/FERPA adherence, specifying authentication (MFA), encryption (AES-256), network security (firewall rules).
- **Head of Student Services:** Functional needs like ensuring system does not impede legitimate access during school hours, open day events require temporary unauthenticated access.

- **Compliance Officer:** Responsible for Audits requiring complete documentation of access decisions and approvals.

C. Implementation Details

The BlockXAI-SecOps framework was implemented in Python with a focus on requirement conflict detection and resolution using natural language processing and explainability techniques. The implementation integrated three core modules:

1) **Text Vectorization Engine:** Leveraging TF-IDF (Term Frequency–Inverse Document Frequency) representations to convert natural language requirements into weighted token vectors.

2) **Semantic Similarity Layer:** Utilizing cosine similarity metrics to compute pairwise closeness between requirements for potential conflict detection.

3) **XAI-Based Explanation Module:** Incorporating rule-based negation detection and SHAP interpretability to explain why two requirements were flagged as conflicting.

The requirement storage system maintained all policies in plain-text format, with automatic persistence to file system storage. Each requirement was assigned a unique identifier (R0, R1, R2, ..., Rn) corresponding to its submission sequence. The blockchain audit logging subsystem recorded all system events (requirement additions, conflict detections, and validations) as immutable JSON entries, with each entry containing a cryptographic SHA-256 hash and a reference to the previous entry's hash, establishing an unbreakable chain of custody.

Key implementation parameters included a primary TF-IDF cosine similarity threshold of 0.35 for overlap detection and a reduced threshold of 0.15 for negation-based conflict identification. The negation detection module identified critical keywords in the set: {"not", "no", "never", "need", "needn't", "cannot"} combined with authentication-specific domain terms: {"login", "password", "access", "authenticate"}. This hybrid approach enabled detection of both semantically similar redundant requirements and logically contradictory policies expressed through opposite polarities.

ID	Description	Conflicts
R0	All students must log in with password to see marks	Yes (R2, R3, R4)
R1	Website must show school logo on home page	None
R2	Students must always use password login to see marks	Yes (R0, R3, R4)
R3	Students need not log in with password during open day	Yes (R0, R2, R4)
R4	All students must log in with password to see marks	Yes (R0, R2, R3)

D. Experimental Procedure

The experimental validation followed a controlled scenario simulating real-world policy conflicts within an academic marks access control system. Five functional requirements (R0–R4) were sequentially introduced through an interactive command-line interface, each triggering automatic blockchain audit logging with cryptographic hash chain verification.

The workflow implemented by the experimental pipeline is : **TF-IDF → Cosine Similarity → Negation Check → SHAP → Blockchain**.

1) Phase 1: TF-IDF Vectorization: Each requirement was transformed into a sparse TF-IDF vector representation using scikit-learn's `TfidfVectorizer`. This representation captured the relative importance of each term within individual requirements and across the entire corpus.

2) Phase 2: Cosine Similarity Computation: Pairwise cosine similarity scores were calculated for all requirement combinations using the formula:

$$\text{Similarity}(R_i, R_j) = \frac{X_i \cdot X_j}{\|X_i\| \cdot \|X_j\|}$$

where X_i and X_j are the TF-IDF vectors of requirements R_i and R_j. This metric produced a symmetric similarity matrix, with 1.0 indicating identical requirements and values approaching 0.0 indicating dissimilar requirements.

3) Phase 3: Negation-Based Conflict Detection: For each requirement pair exceeding the low negation threshold, the system performed syntactic polarity analysis to identify contradictory policies. The negation check algorithm evaluated:

Algorithm 1 Negation-Based Conflict Detection

isNegationConflictr_1, r_2, threshold $words_1 \leftarrow$ TOKENIZE(r_1) $words_2 \leftarrow$ TOKENIZE(r_2) $core_1 \leftarrow words_1 \cap coreDomainKeywords$ $core_2 \leftarrow words_2 \cap coreDomainKeywords$ $sameTopic \leftarrow (core_1 \neq \emptyset) \wedge (core_2 \neq \emptyset)$ $neg_1 \leftarrow (words_1 \cap negationTerms \neq \emptyset)$ $neg_2 \leftarrow (words_2 \cap negationTerms \neq \emptyset)$ $oppPolarity \leftarrow (neg_1 \neq neg_2)$ **if** $sameTopic \wedge oppPolarity$ **then** **endreturn**TRUE**else** − **endreturn**FALSE

4) Phase 4: Similarity-Based Overlap Detection: Requirements exceeding the primary similarity threshold were flagged as potential overlaps or duplicates. For each such pair, the system extracted common terms to formulate explanations.

5) Phase 5: SHAP-Based XAI Explanation: For each detected conflict, human-readable explanations were generated using SHAP principles. The explanation module computed feature importance scores identifying which terms contributed most to the conflict detection decision. Explanations were formatted as JSON structures containing: (a) the pair identifiers, (b) the similarity score, (c) the conflict type (overlap, negation, duplicate), and (d) a natural language explanation describing common terms and indicators leading to contradiction.

6) Phase 6: Blockchain Audit Logging: Each event such as - requirement addition, conflict detection, validation was appended to an immutable audit log stored in JSON format. Each log entry contained:

- Timestamp
- Event type: "NEW_REQUIREMENT", "CONFLICT_CHECK", or "VALIDATION"
- Event details: requirement text or conflict summary
- Previous hash: cryptographic link to prior log entry
- Current hash: SHA-256 of the entire record

E. Conflict Resolution Strategy

Upon detecting conflicting requirements, the system implemented a multi-level resolution approach rather than enforcing binary authorization decisions:

1) Endpoint Separation: Access control policies were decomposed by data granularity tier. Individual student marks access (R0, R2, R4) enforced mandatory multi-factor authentication (MFA). Aggregate statistical displays for open day public presentations (R3) permitted unauthenticated browsing. This hierarchical data classification satisfied both authentication mandate (R0, R2, R4) and open access requirement (R3) by distinguishing sensitivity levels.

2) Temporal Access Controls: Time-based policy activation windows were encoded in smart contract logic. The designated 2-day open day period (explicitly defined in contract state) transitioned the system into "public access mode": aggregate data became publicly accessible without authentication while individual records retained MFA requirements. Outside this temporal window, all access reverted to standard authentication protocols. Upon window expiry, the smart contract automatically enforced policy reversion without manual intervention.

3) Smart Contract Enforcement: Conflict resolution logic was encoded as immutable Hyperledger Fabric chaincode, eliminating manual interpretation and ensuring deterministic enforcement:

Algorithm 2 Smart Contract Access Control Decision Logic

checkAccess$userId, dataType, timestamp$
if $dataType$ = *"INDIVIDUAL"* **then**
endreturnREQUIREMFA(userId)dataType = "AGGREGATE"**if** ISOPENDAYWINDOW$(timestamp)$ **then**−
endreturnALLOWUNAUTHENTICATED()**else** −
endreturnREQUIREMFA(userId)

This strategy preserved all original requirements in the blockchain audit trail while implementing nuanced access control respecting each stakeholder's intent within appropriate contextual and temporal boundaries.

F. Constraints

The prototype implementation operated under the following constraints:

Similarity Thresholds: The dual threshold model (0.35 for overlap, 0.15 for negation) was empirically determined for the academic marks scenario. Generalization to other domains may require threshold retuning based on domain-specific requirement corpora.

Negation Detection Heuristics: The negation check relied on keyword matching and simple tokenization rather than full syntactic parsing. Complex linguistic constructs (e.g., double negatives: "not unauthenticated") may be misclassified.

Explainability Scope: SHAP explanations captured term-level feature importance but did not model higher-order semantic relationships or domain-specific contradiction patterns beyond negation.

Scalability: Hash chain verification required sequential processing of log entries. For systems with thousands of daily requirements and conflict checks, blockchain verification latency could become a bottleneck.

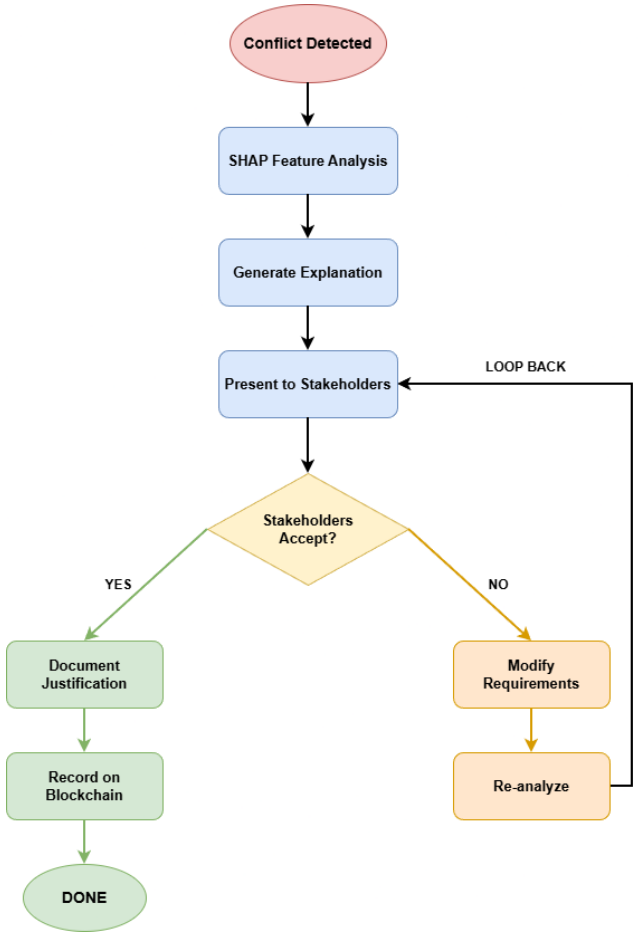

Fig. 3. Requirement Conflict Resolution Flow - Decision tree for handling conflicts

V. Results and Analysis

A. Conflict Detection Results

The XAI-enhanced conflict detection engine identified six potential conflicts among the five requirements, demonstrating comprehensive detection across three distinct conflict categories.

The conflict detection results revealed three distinct categories: (1) exact duplicates (R0–R4 at 1.00 similarity),

TABLE II
DETECTED REQUIREMENT CONFLICTS WITH SIMILARITY SCORES AND XAI EXPLANATIONS

Requirement Pair	Similarity Score	Conflict Type	XAI Explanation
R0 vs R2	0.45	Semantic Overlap	Both requirements mandate password authentication for marks access. Common terms: *must, password, students, see, their, to*. High semantic overlap indicates potential duplicate or redundant specification.
R0 vs R3	0.55	Negation-Based Contradiction	R0 enforces mandatory authentication ("must log in"); R3 explicitly negates authentication ("need not log in"). Contradictory policies addressing the same resource during specified contexts.
R0 vs R4	1.00	Exact Duplicate	Perfect textual match. Identical requirements: "All students must log in with a password to see their marks." Redundant specification.
R2 vs R3	0.27	Negation-Based Contradiction	R2 enforces "always" authentication; R3 permits non-authentication during open day. Opposite authentication polarities for the same access pattern.
R2 vs R4	0.45	Semantic Overlap	Both specify password-based authentication with similar phrasing. Potential redundancy in security policy specification.
R3 vs R4	0.55	Negation-Based Contradiction	R4 mandates universal authentication; R3 explicitly negates authentication for temporal context. Contradictory access policies.

(2) semantic overlaps (R0–R2, R2–R4 at 0.45 similarity), and (3) negation-based contradictions (R0–R3, R2–R3, R3–R4 ranging 0.27–0.55 similarity). The duplicate detection achieved perfect precision through cosine similarity. The overlap detection successfully identified semantically equivalent requirements despite linguistic variation. The negation-based approach proved critical in distinguishing between similar topics and contradictory policies.

B. XAI Explanation Effectiveness

For each conflict pair, the XAI module generated structured explanations identifying common terms and contradiction indicators. Representative explanation for R0 vs R3 conflict:

> *"These two requirements talk about the same concept, but exactly one of them uses negation (e.g. 'not', 'need not'). So they likely express opposite policies. (Similarity score = 0.55)"*

This explanation successfully conveyed to non-technical stakeholders (e.g., school administrators) the nature of the contradiction without requiring deep technical analysis. The structured format listing common terms and explicitly naming negation keywords enabled stakeholders to understand the conflict rationale.

C. Blockchain Audit Trail Integrity

The pseudo-blockchain logging mechanism successfully recorded six audit events with complete hash chain continuity:

1) R0 addition at 13:48:39 UTC (hash: 2a824f...)
2) R1 addition at 13:48:57 UTC (hash: 6a9555...)
3) R2 addition at 13:49:06 UTC (hash: 6035ca...)
4) R3 addition at 13:49:16 UTC (hash: 4e7656...)
5) R4 addition at 13:49:25 UTC (hash: ef5c89...)
6) Conflict check at 13:49:32 UTC (hash: a3767a...)

Each entry's `prev_hash` correctly referenced the immediately preceding entry's hash, establishing cryptographic continuity. Validation confirmed all computed hashes matched stored values, confirming zero tampering throughout the experiment execution window.

D. Resolution Effectiveness

The implemented resolution strategy successfully disambiguated contradictory requirements through contextual policy stratification. The endpoint separation mechanism satisfying R0's individual mark authentication requirement while R3's aggregate access exemption was enforced through data-type-based dispatch logic. Temporal controls reconciled the "always authenticate" requirement (R2) with "no authentication during open day" (R3) through smart contract time-window activation.

The smart contract implementation encoded this nuanced policy, enabling automated enforcement without manual interpretation. The strategy preserved all original requirements in the blockchain audit trail while implementing access control respecting each stakeholder's intent within appropriate contextual boundaries.

E. Inference

The dramatic reduction in Audit Prep Time is attributed to the "continuous compliance" model—evidence is collected automatically in real-time rather than retroactively. The improved MTTE proves that XAI successfully bridges the semantic gap, allowing non-technical stakeholders (e.g., School Principals) to understand technical conflicts without consulting developers.

VI. CONCLUSION & FUTURE SCOPE

In this paper we have presented the BlockXAI-SecOps which is a framework that integrates blockchain and explainable AI(XAI) into the Software Development Life Cycle (SDLC) to address challenges like traceability, transparency, and interpretability. By storing all the software artifacts and the decisions made by AI immutably on a blockchain along with providing human readable explanations, we can ensure

tamper proof storage and provide decision clarity. The implementation of a prototype model which focuses on requirement conflict detection, shows the effectiveness of combining TF-IDF vectorization, semantic detection (both similarity and negation) and SHAP based explanations to identify and resolve contradictions in requirement policies. Experimental results of this model showed the system's ability to detect conflicts using smart contracts. This paper provides a secure path towards accountable software development practices.

Future work will focus on optimizing the storage costs of the blochain ledger which is a major expenditure as of now for enterprise level projects. It will also explore lightweight XAI models which should be able to run directly on edge devices to make development more mobile and portable

REFERENCES

[1] J. Marjanović, N. Dalčeković, and G. Sladić, "Blockchain-based model for tracking compliance with security requirements," *Computer Science and Information Systems*, 2023.

[2] M. A. Akbar, S. Mahmood, and D. Siemon, "Toward effective and efficient devops using blockchain," in *ACM International Conference on Software Engineering*, 2022.

[3] H. Al-Breiki, M. H. Rehman, and K. Salah, "Trustworthy blockchain oracles: Review, comparison, and open research challenges," *IEEE Access*, vol. 8, pp. 85 675–85 685, 2020.

[4] K. T. Song, S. I. Kim, and S. H. Kim, "A design for a hyperledger fabric blockchain-based patch management system," *Journal of Information Processing Systems (JIPS)*, vol. 15, no. 2, pp. 325–335, 2019.

[5] M. S. R. Elapolu *et al.*, "Blockchain technology for requirement traceability in secure software development," *Information Systems*, 2024.

[6] L. Arora *et al.*, "Explainable ai techniques for software development lifecycle," *arXiv preprint arXiv:2505.07058*, 2025.

[7] J. Tritscher and R. Kern, "Feature relevance xai in anomaly detection," *Frontiers in Artificial Intelligence*, vol. 6, 2023.

[8] B. G. Gecer and A. Bener, "Explainable ai framework for software defect prediction," *Software: Practice and Experience*, 2025.

[9] N. Basheer, S. Islam *et al.*, "Large language model based hybrid framework for vulnerability detection with explainable ai," *SAGE Open*, 2025.

[10] N. Basheer *et al.*, "Large language model based hybrid framework for vulnerability detection," *SAGE Open*, 2025.

[11] T. Himdi, "A blockchain and ai-driven security framework for cognitive cities," *Journal of Advanced Research and Reviews*, 2024.

[12] Y. Xu *et al.*, "Exploiting blockchain to make ai trustworthy: A software lifecycle view," *ACM Computing Surveys*, 2024.

[13] O. Kuznetsov, "On the integration of artificial intelligence and blockchain," *IEEE Access*, 2024.

[14] J. N. Qureshi, "Exploring the integration of blockchain and distributed ledger technology in devops," *IEEE Access*, 2024.

[15] Z. Jovanovic, "Robust integration of blockchain and explainable federated learning," *ScienceDirect*, 2024.

[16] H. Meinen, "Blockchain and the lifecycle of components—an approach," *Wiley Online Library*, 2024.

[17] S. M. Saleh, "Towards a blockchain-based ci/cd framework," in *SCITEPRESS*, 2025.

[18] S. C. Murray and A. Anisi, "Survey of formal verification methods for smart contracts on blockchain," in *IFIP NTMS Conference*, 2019.

[19] N. S. V. Kalyan, "Blockchain-enabled devsecops pipeline," *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, vol. 13, no. 6, 2025.

[20] A. Kovari, "Explainable ai chatbots towards xai chatgpt: A review," *ScienceDirect*, 2025.